

Sanos y Salvos

Informe de Implementación Tecnológica

Institución: Duoc UC

Asignatura: Desarrollo Full Stack III

Caso: Sanos y Salvos – Plataforma Inteligente para la localización y recuperación de mascotas perdidas

Integrantes:

- Juan Vargas Castillo
- Matilda Vargas Canseco

Sección: 001D

Fecha: 30 de junio de 2026.

1. Introducción

El presente informe describe las mejoras tecnológicas implementadas en el proyecto Sanos y Salvos, una plataforma desarrollada bajo una arquitectura de microservicios cuyo propósito es facilitar el registro de mascotas perdidas y encontradas, permitiendo además la gestión de usuarios, el almacenamiento de ubicaciones geográficas y la detección de coincidencias entre publicaciones.

Durante esta evaluación se realizaron diversas mejoras orientadas a fortalecer la arquitectura del sistema, incrementar su mantenibilidad y optimizar su funcionamiento. Entre los principales cambios se encuentra la incorporación de un API Gateway como punto único de acceso a los microservicios, la implementación de un Circuit Breaker para mejorar la tolerancia a fallos durante la comunicación entre servicios, la actualización de la interfaz administrativa del frontend y la corrección de diversos errores funcionales detectados en versiones anteriores.

Adicionalmente, se integró documentación mediante Swagger/OpenAPI, se consolidó el despliegue del sistema utilizando Docker y Docker Compose, y se verificó el correcto funcionamiento de todas las funcionalidades implementadas tanto en el frontend como en el backend.

El objetivo de este informe es presentar la arquitectura tecnológica del proyecto, describir las tecnologías empleadas y documentar las mejoras incorporadas durante el desarrollo de esta evaluación, evidenciando su funcionamiento mediante capturas del sistema y de los componentes principales.

2. Arquitectura General del Sistema

2.1 Descripción de la arquitectura

El proyecto Sanos y Salvos fue desarrollado siguiendo una arquitectura basada en microservicios, donde cada componente del sistema cumple una responsabilidad específica e independiente. Esta arquitectura permite desacoplar la lógica del negocio en distintos servicios, facilitando su mantenimiento, escalabilidad y evolución.

La solución está compuesta por un frontend desarrollado en React, el cual actúa como interfaz para los usuarios del sistema. Todas las solicitudes realizadas desde el cliente son enviadas al API Gateway, encargado de centralizar el acceso al backend y enrutar cada petición hacia el microservicio correspondiente.

Cada microservicio administra un dominio funcional específico del sistema:

- Microservicio de Usuarios: encargado del registro, autenticación y administración de usuarios.
- Microservicio de Mascotas: responsable de la gestión de mascotas perdidas y encontradas.
- Microservicio de Coincidencias: analiza la información disponible para determinar posibles coincidencias entre mascotas reportadas.
- Microservicio de Geolocalización: administra la información geográfica y realiza cálculos de distancia utilizados durante el proceso de coincidencias.

Los microservicios que requieren persistencia utilizan bases de datos independientes en MySQL, mientras que la comunicación entre ellos se realiza mediante OpenFeign, incorporando mecanismos de tolerancia a fallos mediante Resilience4J Circuit Breaker.

Finalmente, todo el entorno puede ejecutarse mediante Docker Compose, permitiendo levantar de forma automática el API Gateway, los microservicios, las bases de datos y las herramientas de apoyo utilizadas durante el desarrollo.

Estructura general del proyecto Backend.

```

✓ SANOS-Y-SALVOS-BACKEND-VARGAS-VARGAS
  > .github
  > bff-sanosysalvos
  > gateway-sanosysalvos
  > ms-coincidencias
  > ms-geolocalizacion
  > ms-mascotas
  > ms-usuarios
  > .dockerignore
  > .gitignore
  > docker-compose.yml
  > README.md
  
```

Docker Desktop con los contenedores levantados.


sanos-y-salvos-backend-vargas-vargas
C:\Users\mati\OneDrive\Escritorio\sanos-y-salvos-backend-vargas-vargas
View cont

gateway-san... sanos-y-salvos-ba 8080:8080	<pre> o.s.core.events.SpringDocAppInitializer : SpringDoc /v3/api-doc To disable it in production, set the property 'springdoc.api-doc 2026-06-29T23:50:14.834Z WARN 1 --- [ms-geolocalizacion] [o.s.core.events.SpringDocAppInitializer : SpringDoc /swagger-u default. To disable it in production, set the property 'springdoc </pre>
ms-coinciden... sanos-y-salvos-ba 8083:8083	<pre> 2026-06-29T23:50:14.943Z INFO 1 --- [ms-mascotas] [c.s.mascotas.MsMascotasApplication : Started MsMascotasApp (process running for 45.788) 2026-06-29T23:50:14.995Z WARN 1 --- [ms-mascotas] [o.s.core.events.SpringDocAppInitializer : SpringDoc /v3/api-doc To disable it in production, set the property 'springdoc.api-doc 2026-06-29T23:50:14.998Z WARN 1 --- [ms-mascotas] [o.s.core.events.SpringDocAppInitializer : SpringDoc /swagger-u default. To disable it in production, set the property 'springdoc </pre>
ms-usuarios sanos-y-salvos-ba 8081:8081	<pre> 2026-06-29T23:50:15.136Z INFO 1 --- [ms-usuarios] [o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on poi ''' 2026-06-29T23:50:15.232Z INFO 1 --- [ms-usuarios] [c.s.usuarios.MsUsuariosApplication : Started MsUsuariosApp (process running for 46.077) 2026-06-29T23:50:15.254Z WARN 1 --- [ms-usuarios] [o.s.core.events.SpringDocAppInitializer : SpringDoc /v3/api-doc To disable it in production, set the property 'springdoc.api-doc 2026-06-29T23:50:15.255Z WARN 1 --- [ms-usuarios] [o.s.core.events.SpringDocAppInitializer : SpringDoc /swagger-u default. To disable it in production, set the property 'springdoc </pre>
ms-mascotas sanos-y-salvos-ba 8082:8082	<pre> 2026-06-29 23:53:46.698 UTC [27] LOG: checkpoint starting: tim 2026-06-29 23:53:47.835 UTC [27] LOG: checkpoint complete: wro file(s) added, 0 removed, 0 recycled; write=1.112 s, sync=0.010 longest=0.005 s, average=0.001 s; distance=10 kB, estimate=10 ki </pre>
ms-geocali... sanos-y-salvos-ba 8084:8084	
sonarqube sonarqube:its-con 9000:9000	
sonarqube-db postgres:15-alpine 5432:5432	
mysql mysql:8.0 3306:3306	

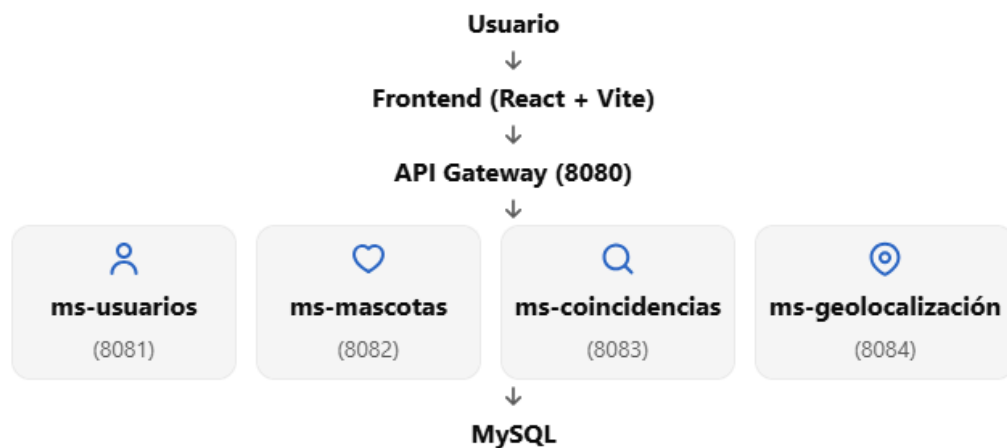
2.2 Flujo general del sistema

El flujo de funcionamiento del sistema comienza cuando un usuario interactúa con la interfaz desarrollada en React. Las solicitudes HTTP generadas por el frontend son enviadas al API Gateway, el cual actúa como único punto de entrada hacia el backend.

El Gateway analiza la ruta solicitada y redirige la petición al microservicio correspondiente. Dependiendo de la operación realizada, los microservicios pueden acceder a sus respectivas bases de datos o comunicarse entre sí para obtener información adicional.

Por ejemplo, durante la búsqueda de coincidencias, el microservicio de Coincidencias consulta información de mascotas y utiliza el microservicio de Geolocalización para calcular distancias entre reportes, aplicando un Circuit Breaker para garantizar la continuidad del servicio ante posibles fallos.

Este modelo permite mantener un bajo acoplamiento entre los componentes del sistema y facilita la incorporación de nuevos servicios sin afectar el funcionamiento del resto de la aplicación.



3. Tecnologías utilizadas

3.1 Tecnologías del Backend

El backend del proyecto fue desarrollado utilizando Java 17 y el framework Spring Boot, adoptando una arquitectura basada en microservicios. Cada servicio fue implementado como una aplicación independiente, permitiendo separar las responsabilidades del sistema y facilitar su mantenimiento.

Para la exposición de servicios REST se utilizó Spring Web, mientras que la persistencia de datos fue implementada mediante Spring Data JPA y Hibernate, utilizando MySQL como sistema gestor de bases de datos para los microservicios que requieren almacenamiento de información.

La autenticación y autorización fueron implementadas mediante Spring Security utilizando JSON Web Tokens (JWT), permitiendo proteger los distintos endpoints del sistema mediante un modelo de autenticación sin estado (*stateless*).

La comunicación entre microservicios se realizó utilizando OpenFeign, simplificando el consumo de servicios remotos mediante clientes declarativos. Adicionalmente, el microservicio de Coincidencias incorpora Resilience4J, implementando el patrón Circuit Breaker para mejorar la disponibilidad del sistema frente a fallos de comunicación.

Como punto único de entrada al backend se implementó Spring Cloud Gateway, encargado de centralizar el acceso a los distintos microservicios.

Finalmente, la documentación de las APIs se realizó mediante Swagger/OpenAPI, permitiendo visualizar y probar los distintos endpoints disponibles en cada microservicio.

3.2 Tecnologías del Frontend

El frontend fue desarrollado utilizando React junto con Vite, proporcionando una interfaz moderna, rápida y modular para los usuarios del sistema.

La navegación entre las distintas vistas se implementó mediante React Router DOM, permitiendo separar las rutas públicas, privadas y administrativas.

Las solicitudes HTTP hacia el backend se centralizaron utilizando Axios, configurando una instancia común que incorpora automáticamente el token JWT en el encabezado de autorización para todas las peticiones autenticadas.

Para la administración del estado global de autenticación se utilizó React Context API, almacenando la información de la sesión del usuario y controlando el acceso a las distintas funcionalidades mediante componentes de protección de rutas.

La interfaz incorpora además Bootstrap como apoyo para el diseño responsivo de la aplicación y Leaflet, junto con React Leaflet, para representar visualmente la ubicación geográfica de las mascotas mediante mapas interactivos.

3.3 Herramientas de apoyo

Durante el desarrollo del proyecto también se utilizaron diversas herramientas orientadas a facilitar el despliegue, la documentación y la evaluación de la calidad del software.

La contenerización de los servicios se realizó mediante Docker, mientras que Docker Compose permitió orquestar el levantamiento completo del entorno de desarrollo, incluyendo los microservicios, el API Gateway, las bases de datos y las herramientas auxiliares.

Para el análisis estático del código se incorporó SonarQube, permitiendo evaluar métricas de calidad, mantenibilidad y posibles problemas de seguridad.

Adicionalmente, se utilizó JaCoCo para generar reportes de cobertura de pruebas automatizadas, facilitando la medición del porcentaje de código ejercitado durante la ejecución de las pruebas.

4. Aplicación de patrones de diseño

Durante el desarrollo del proyecto se aplicaron distintos patrones de diseño y arquitectónicos con el objetivo de mejorar la organización del código, facilitar su mantenimiento y promover la reutilización de componentes tanto en el backend como en el frontend.

En el backend se implementó el patrón Repository, permitiendo abstraer el acceso a la base de datos mediante Spring Data JPA. Gracias a este patrón, la lógica de persistencia quedó separada de la lógica de negocio, facilitando futuras modificaciones en la capa de datos sin afectar el resto del sistema.

Asimismo, se utilizó el patrón Service Layer, donde cada microservicio concentra la lógica de negocio dentro de clases de servicio independientes de los controladores REST. Esto permitió mantener una clara separación de responsabilidades y un código más modular y reutilizable.

Para el intercambio de información entre los distintos componentes también se implementaron DTO (Data Transfer Object), evitando exponer directamente las entidades de la base de datos y permitiendo controlar la información enviada y recibida por las APIs.

También se incorporó el patrón Factory Method en la creación de objetos dentro del sistema, centralizando el proceso de instanciación y permitiendo desacoplar la lógica de creación respecto de las clases que utilizan dichos objetos. Esta decisión facilitó la extensión del proyecto y redujo la dependencia entre componentes.

Como patrón arquitectónico se implementó un API Gateway, encargado de centralizar todas las solicitudes provenientes del frontend y redirigirlas al microservicio correspondiente. Esta decisión permitió desacoplar el cliente de la arquitectura interna del backend y facilitar futuras ampliaciones del sistema.

Adicionalmente, se implementó el patrón Circuit Breaker mediante Resilience4J para proteger la comunicación entre microservicios ante posibles fallos, evitando errores en cascada y mejorando la disponibilidad general de la plataforma.

En el frontend también se aplicaron principios de modularidad mediante la separación de componentes reutilizables, servicios HTTP, rutas protegidas y administración centralizada del estado utilizando React Context API, facilitando el mantenimiento y la evolución de la aplicación.

En conjunto, la utilización de estos patrones permitió construir una solución más organizada, escalable y mantenible, reduciendo el acoplamiento entre componentes, favoreciendo la reutilización del código y facilitando futuras modificaciones al sistema.

5. Estrategia de Branching y Gestión de Versiones

Durante el desarrollo del proyecto se utilizó una estrategia de branching inspirada en **GitHub Flow**, permitiendo que cada integrante trabajara de manera independiente sobre nuevas funcionalidades, correcciones de errores y mejoras en la documentación, reduciendo conflictos durante el desarrollo.

Para mantener organizado el repositorio se definió una convención de nombres para las ramas, identificando tanto el tipo de trabajo realizado como el integrante responsable de la implementación.

Se utilizaron principalmente los siguientes tipos de ramas:

- **feature/** para el desarrollo de nuevas funcionalidades.
- **fix/** para la corrección de errores detectados durante el desarrollo.
- **docs/** para modificaciones en la documentación del proyecto.
- **refactor/** para reorganizar código sin alterar su funcionamiento.
- **test/** para la implementación de pruebas unitarias cuando fue necesario.

Además, cada rama incorporaba el nombre del integrante que realizó el trabajo, facilitando el seguimiento de los cambios y la identificación del responsable de cada implementación.

Una vez finalizada cada tarea, los cambios eran integrados mediante **Pull Requests**, permitiendo revisar el código antes de incorporarlo a la rama principal. Esta estrategia facilitó el trabajo colaborativo, redujo conflictos de integración y permitió mantener un historial de versiones claro y organizado durante todo el desarrollo del proyecto.

☐		0 Open	☒	99 Closed	Author	Label	Projects	Milestones	Reviews	Assignee	Sort
☐		fix: se actualiza el readme y queda finalizado	#99	by MatildaVargasC	was merged 8 hours ago						Review required
☐		Docs: Actualizo el readme con todo lo nuevo implementado	#98	by MatildaVargasC	was merged yesterday						Review required
☐		feat: permito acceso a Swagger UI y OpenAPI en los microservicios	#97	by MatildaVargasC	was merged yesterday						Review required
☐		feature: se agrega perfectamente el swager con sus dependencias a tod...	#96	by MatildaVargasC	was merged yesterday						Review required
☐		feat(coincidencias): implemento Circuit Breaker con Resilience4j	#95	by MatildaVargasC	was merged yesterday						Review required
☐		agrego depedencia en coincidencias para circuit breaker	#94	by MatildaVargasC	was merged yesterday						Review required
☐		se agrega con exito cors en gateway	#93	by juavargasc-del	was merged last week						Review required
☐		fix(gateway): configurar CORS para acceso desde frontend	#92	by juavargasc-del	was merged last week						Review required
☐		refactor: reemplazamos el bff por el api gateway	#91	by juavargasc-del	was merged last week						Review required
☐		feat(gateway):se agrega dockerfile a gateway	#90	by juavargasc-del	was merged last week						Review required
☐		Feature/juan vargas api gateway	#89	by juavargasc-del	was merged last week						Review required
☐		Doc: se adjunta readme del backend	#88	by juavargasc-del	was merged 2 weeks ago						Review required

fix: se actualiza el readme y queda finalizado #99

Merged juavargasc-del merged 1 commit into `main` from `fix/readme-matilda` 8 hours ago

Conversation **0** Commits **1** Checks **0** Files changed **1**

MatildaVargasC commented 8 hours ago

No description provided.

fix: se actualiza el readme y queda finalizado 97b10a5

juavargasc-del merged commit **315569f** into `main` 8 hours ago Revert

juavargasc-del deleted the `fix/readme-matilda` branch 4 hours ago Restore branch

se agrega con exito cors en gateway #93

 **Merged** juavargasc-del merged 1 commit into `main` from `feature/integracion-clase-cors-gateway-juan`  last week

 Conversation **0**  Commits **1**  Checks **0**  Files changed **2**



juavargasc-del commented [last week](#) 

No description provided.



 se agrega con exito cors en gateway

[c1c77ac](#)



 juavargasc-del merged commit **91facc7** into `main` [last week](#)

[Revert](#)



 juavargasc-del deleted the `feature/integracion-clase-cors-gateway-juan` branch [yesterday](#)

[Restore branch](#)



Pull request successfully merged and closed

You're all set — the branch has been merged.

6. Cambios Implementados para la Evaluación

6.1 Implementación del API Gateway

Durante el desarrollo de esta evaluación se incorporó un API Gateway como punto único de acceso al backend, reemplazando el acceso directo que anteriormente realizaba el frontend hacia los distintos microservicios.

La incorporación del Gateway permitió centralizar el ingreso de todas las solicitudes HTTP, simplificando la comunicación entre el cliente y los servicios internos del sistema. De esta manera, el frontend únicamente necesita conocer la dirección del Gateway, mientras que este se encarga de redirigir automáticamente cada petición hacia el microservicio correspondiente según la ruta configurada.

Además de facilitar la administración del sistema, esta arquitectura reduce el acoplamiento entre el frontend y los microservicios, permitiendo modificar direcciones, agregar nuevos servicios o cambiar la infraestructura interna sin afectar al cliente.

La implementación fue realizada utilizando Spring Cloud Gateway, definiendo reglas de enrutamiento para los distintos microservicios del proyecto.

Beneficios obtenidos

- Centralización del acceso al backend.
- Menor acoplamiento entre frontend y microservicios.
- Mayor facilidad para incorporar nuevos servicios.
- Mejor mantenibilidad de la arquitectura.
- Punto único para futuras políticas de seguridad, monitoreo y balanceo de carga

6.2 Implementación del Circuit Breaker

Con el objetivo de aumentar la disponibilidad del sistema frente a posibles fallos de comunicación entre microservicios, se implementó el patrón Circuit Breaker utilizando la librería Resilience4J.

Esta funcionalidad fue incorporada en el microservicio de Coincidencias, el cual realiza consultas al microservicio de Geolocalización para calcular distancias entre mascotas reportadas.

El Circuit Breaker monitorea continuamente el estado de las llamadas remotas y, cuando detecta múltiples fallos consecutivos, interrumpe temporalmente las solicitudes hacia el servicio afectado. Esto evita tiempos de espera excesivos y disminuye el impacto de una posible caída sobre el resto del sistema.

Además, se implementó un método de recuperación (*fallback*) que permite responder de manera controlada cuando el servicio remoto no se encuentra disponible.

Beneficios obtenidos

- Mayor tolerancia a fallos.
- Protección frente a caídas de servicios externos.
- Reducción de tiempos de espera.
- Mayor estabilidad general del sistema.

```
ms-coincidencias > src > main > java > com > sanosysalvos > coincidencias > service > impl > J CoincidenciaServiceImpl.java > Language Support for Java(TM) by Red Hat > Co

27 public class CoincidenciaServiceImpl implements CoincidenciaService {
162
    Tabnine | Edit | Test | Explain | Document
163     @CircuitBreaker(name = "geolocalizacionCircuitBreaker", fallbackMethod = "fallbackObtenerDistanciaEntreMascotas")
164     public DistanciaResponseDTO obtenerDistanciaEntreMascotas(Long mascotaPerdidaId, Long mascotaEncontradaId) {
165         UbicacionMascotaDTO ubicacionPerdida = geolocalizacionFeignClient.obtenerUbicacionPorMascota(mascotaPerdidaId);
166         UbicacionMascotaDTO ubicacionEncontrada = geolocalizacionFeignClient.obtenerUbicacionPorMascota(mascotaEncontradaId);
167
168         if (ubicacionPerdida == null || ubicacionEncontrada == null) {
169             return null;
170         }
171
172         return geolocalizacionFeignClient.calcularDistancia(
173             ubicacionPerdida.getLatitud(),
174             ubicacionPerdida.getLongitud(),
175             ubicacionEncontrada.getLatitud(),
176             ubicacionEncontrada.getLongitud()
177         );
178     }
179 }
```

6.3 Mejoras en la Vista Administrativa

Durante esta evaluación también se realizaron mejoras sobre el panel de administración del frontend, corrigiendo diversos errores presentes en versiones anteriores e incorporando funcionalidades que anteriormente no se encontraban completamente operativas.

Se revisó el funcionamiento de los distintos módulos administrativos, asegurando el correcto consumo de los servicios expuestos por el API Gateway y mejorando la interacción con los microservicios del backend.

Como resultado, el administrador puede gestionar correctamente la información del sistema mediante una interfaz unificada, accediendo a los distintos módulos destinados a la administración de usuarios, mascotas y coincidencias.

Mejoras realizadas

- Corrección de errores de navegación.
- Corrección del consumo de servicios.
- Integración completa con el API Gateway.
- Mejoras visuales en la interfaz administrativa.
- Validación del funcionamiento de todas las funcionalidades implementadas.

6.4 Corrección de errores y funcionalidades

Como parte del proceso de integración final del sistema, se realizó una revisión completa de las funcionalidades implementadas tanto en el frontend como en el backend.

Durante esta etapa se corrigieron diversos errores relacionados con la autenticación mediante JWT, el acceso a rutas protegidas, la comunicación entre microservicios y la correcta visualización de la información en la interfaz.

Asimismo, se verificó el funcionamiento de las operaciones principales del sistema, incluyendo el registro y autenticación de usuarios, la administración de mascotas, la consulta de coincidencias y las operaciones de geolocalización, asegurando la correcta interacción entre todos los componentes de la arquitectura.

Estas correcciones permitieron entregar una versión estable y completamente funcional de la plataforma.

6.5 Documentación de APIs mediante Swagger

Con el propósito de facilitar el desarrollo, las pruebas y la comprensión del sistema, se incorporó documentación automática mediante Swagger/OpenAPI en los microservicios desarrollados con Spring Boot.

Cada microservicio expone su propia documentación, permitiendo visualizar los endpoints disponibles, los parámetros requeridos, las respuestas esperadas y realizar pruebas directamente desde la interfaz web proporcionada por Swagger.

Esta documentación facilita la integración entre componentes y mejora considerablemente el proceso de validación de las funcionalidades implementadas.

Swagger Usuarios como ejemplo:

 Swagger
Powered by SWAGTBEAR

Explore



OpenAPI definition v0 OAS 3.1

/v3/api-docs

Servers

Autenticación Operaciones de inicio de sesión del microservicio de usuarios ^

POST

/api/auth/login

Inicio sesión

▼

Usuarios Gestión de usuarios del sistema ^

CET

/api/usuarios

Listar usuarios

▼

POST

/api/usuarios

Crear usuario

▼

POST

/api/usuarios/login

Autenticar usuario

▼

CET

/api/usuarios/{id}

Consultar usuario por ID

▼

DELETE

/api/usuarios/{id}

Eliminar usuario

▼

7. Conclusiones

El desarrollo realizado durante esta evaluación permitió fortalecer significativamente la arquitectura del proyecto Sanos y Salvos, incorporando mejoras que incrementan su escalabilidad, mantenibilidad y disponibilidad.

La implementación del API Gateway permitió centralizar el acceso a los microservicios, simplificando la comunicación entre el frontend y el backend y reduciendo el acoplamiento entre los distintos componentes del sistema.

Asimismo, la incorporación del patrón Circuit Breaker mediante Resilience4J aumentó la tolerancia a fallos durante la comunicación entre microservicios, contribuyendo a mejorar la estabilidad de la plataforma frente a posibles interrupciones de los servicios internos.

En el frontend se realizaron diversas mejoras funcionales y visuales, especialmente en el panel administrativo, corrigiendo errores detectados y asegurando el correcto funcionamiento de las principales funcionalidades del sistema.

Finalmente, la utilización de herramientas como Docker, Docker Compose, Swagger/OpenAPI, SonarQube y JaCoCo permitió facilitar el despliegue, la documentación y la evaluación de la calidad del software, entregando una solución más robusta y preparada para futuras ampliaciones.

En conjunto, las mejoras implementadas permitieron consolidar una arquitectura moderna basada en microservicios, cumpliendo los objetivos planteados para la presente evaluación y dejando una base sólida para la evolución futura del proyecto.

